

## Project Design Phase-II

### Technology Stack (Architecture & Stack)

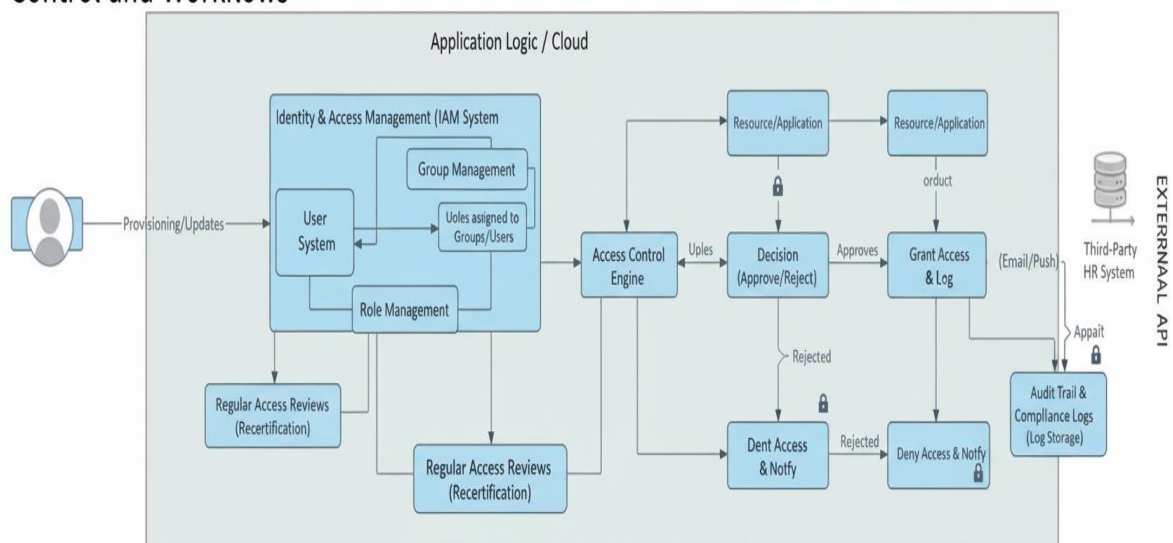
|                      |                                                                               |
|----------------------|-------------------------------------------------------------------------------|
| <b>Date</b>          | 31 October 2025                                                               |
| <b>Team ID</b>       | <b>NM2025TMID02027</b>                                                        |
| <b>Project Name</b>  | Optimizing User, Group, and Role Management with Access Control and Workflows |
| <b>Maximum Marks</b> | 4 Marks                                                                       |

### Technical Architecture

The project architecture focuses on creating a **secure, automated, and scalable environment** for managing users, groups, and roles through **access control and workflow automation**.

It combines frontend interfaces, workflow engines, backend logic, and access control layers to ensure secure and efficient role management.

### Optimizing User, Group, and Role Management with Access Control and Workflows



**Table 1: Components & Technologies**

| S.No | Component Description                                                                                                      | Technology Used                                     |
|------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| 1.   | <b>User Interface (UI)</b> – Admin and users interact through a responsive dashboard for user, group, and role operations. | React.js / HTML / CSS / JavaScript                  |
| 2.   | <b>Application Logic Layer – 1:</b> Handles user creation, validation, and group-role assignment.                          | Node.js / Express.js                                |
| 3.   | <b>Application Logic Layer – 2:</b> Implements dependency validation between user, group, and role relationships.          | Server-side scripting (JavaScript / Node.js)        |
| 4.   | <b>Workflow Engine:</b> Automates approval processes for user access requests.                                             | ServiceNow Flow Designer / Node Workflow Library    |
| 5.   | <b>Access Control Engine (RBAC):</b> Enforces access restrictions and role-based permissions dynamically.                  | Role-Based Access Control (RBAC) Logic / Middleware |
| 6.   | <b>Database Layer:</b> Stores user profiles, group details, roles, and access history.                                     | MySQL / PostgreSQL                                  |
| 7.   | <b>Cloud Infrastructure:</b> Provides scalability, hosting, and backup support for the application.                        | AWS / Azure Cloud Services                          |
| 8.   | <b>API Gateway:</b> Facilitates communication between frontend, backend, and external integrations.                        | RESTful APIs                                        |
| 9.   | <b>Audit and Logging Service:</b> Tracks workflow and access control events for monitoring and compliance.                 | MongoDB / Elasticsearch / ServiceNow Logs           |
| 10.  | <b>Notification System:</b> Sends alerts and status updates after workflow approval or access changes.                     | NodeMailer / Firebase Cloud Messaging (FCM)         |
| 11.  | <b>External Integration (Optional):</b> Connects with third-party HRMS or LDAP for user verification.                      | LDAP Integration / REST API                         |
| 12.  | <b>Infrastructure (Deployment):</b> Deployed on scalable cloud environments for continuous availability.                   | AWS EC2 / Azure App Service                         |

**Table 2: Application Characteristics**

| S.No | Characteristic                  | Description                                                                                           | Technology / Implementation                            |
|------|---------------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 1.   | <b>Open-Source Frameworks</b>   | Uses open-source frameworks for scalability and ease of customization.                                | Node.js, React.js                                      |
| 2.   | <b>Security Implementations</b> | Incorporates Role-Based Access Control (RBAC), secure session handling, and encrypted communications. | JWT Authentication, ACLs, HTTPS                        |
| 3.   | <b>Scalable Architecture</b>    | Built with a modular and microservice-ready structure to handle large-scale enterprise workloads.     | Node.js, AWS Auto Scaling                              |
| 4.   | <b>Availability</b>             | High availability ensured through load balancing and failover redundancy.                             | AWS Elastic Load Balancer / Azure Cloud Load Balancing |
| 5.   | <b>Performance</b>              | Optimized through asynchronous workflows, caching, and efficient database indexing.                   | Redis Cache / Indexed SQL Queries                      |
| 6.   | <b>Auditability</b>             | Each user action, role change, or workflow approval is logged for compliance and traceability.        | MongoDB Logs / CloudWatch                              |
| 7.   | <b>Maintainability</b>          | Designed with modular components for easy updates and debugging.                                      | Docker Containers / CI-CD Pipelines                    |
| 8.   | <b>Interoperability</b>         | Supports integration with identity management systems and third-party applications.                   | REST APIs / LDAP / OAuth 2.0                           |